

Addivox

version 1.0.1 documentation

Table of contents

1. Introduction	3
2. Getting Started	4
2.1 Demo version	4
2.2 Purchasing	5
2.3 Installation – macOS	6
2.4 Installation – Windows	8
3. Using Addivox	9
3.1 Main UI	9
3.2 Patches	12
3.3 EQ	16
3.4 Per-harmonic Settings	17
3.5 Key Notes and Interpolation	19
3.6 Pitch settings	20
3.7 Global settings	21
3.8 Effects	22
4. Support	23
4.1 Videos	23
4.2 FAQ and Miscellaneous Tips	24
4.3 Support and feedback	26
4.4 Release notes	27
4.5 Known issues	29
4.6 License	30

1. Introduction



Addivox is an additive synthesizer for wind controllers. Key features include:

- Additive synthesis, creating sounds by summing harmonics.
- Monosynth, plays one note at a time.
- Per-harmonic controls over level, breath response, attack, release, pitch, pan and more.
- Designed specifically for wind controllers.
- EQ shaping to create formants.
- Built-in effects.
- Available for macOS and Windows (iOS version coming soon).
- Can run as a standalone instrument or as a plugin in a DAW.
- Built using the [iPlug2](#) framework.

Check out the [Videos](#) page for some demonstrations and tutorials. You can also download a [Demo version](#) of Addivox to try. If you like it, head over to [Purchasing](#) to buy Addivox.

The online version of this documentation is available at <https://rrwick.github.io/Addivox/>.

2. Getting Started

2.1 Demo version

The demo version of Addivox is available for all of the same app and plugin formats as the full version of Addivox.

It has full functionality except:

- It will only play white-key notes (e.g. C major).
- Transposition is disabled (since that could be a partial workaround of the white-note-only limitation).

Before purchasing Addivox, I would encourage you to try the demo version. This will let you test whether it works on your computer and DAW, and whether you like the sound.

You can download the demo version of Addivox from the [GitHub releases page](#), and here are direct links to the zip files:

`AddivoxDemo_v1.0.1_macOS.zip`

`AddivoxDemo_v1.0.1_Windows.zip`

After downloading, see the [Installation – macOS](#) or [Installation – Windows](#) page for instructions on how to install the demo version of Addivox.

After purchasing

The demo version is named `AddivoxDemo`, so it can be installed alongside the full version of `Addivox` without the two conflicting. After buying the full version, however, most users will probably want to uninstall the demo version.

If you created custom patches in the demo version, copy them from the demo patch folder to the full-version patch folder. See [Patches](#) for the patch folder locations.



2.2 Purchasing

Before purchasing

Before purchasing, try the [demo version of Addivox](#) to make sure you like the sound and that Addivox works well on your system.

Full version

[Buy the full version of Addivox](#)

The full version of Addivox is US\$50 as a one-time purchase. It includes the standalone app and plugin formats for all supported desktop platforms. After placing an order via the Lemon Squeezy store, you will see a "View Order and Download Files" button which will also be emailed to you. Clicking this will give you access to the macOS and Windows full version Addivox files.

A single purchase gives one user access to Addivox on all of their own devices. Future updates to the full version are included at no extra cost – just return to your order (via the link in your confirmation email or by logging in at lemonsqueezy.com/my-orders with the email you purchased with) and download the latest version.

The full version of Addivox does not use DRM, activation codes or online licence checks. After purchase, you can install it on your own computers without needing to reactivate it. Please do not share the paid download files with others. Addivox is a small independent project, and sales help support future development, updates and documentation.

iOS version

Addivox for the iPad is coming soon. Stay tuned!

Supporting iPlug2

Addivox is built with [iPlug2](#), a free and open-source audio plugin framework. If you enjoy Addivox, please also consider supporting iPlug2 through its [GitHub Sponsors page](#).

2.3 Installation – macOS

Addivox for macOS is delivered as a zip file with a name like `Addivox_v1.0.0_macOS.zip`. After downloading it, double-click to unzip it. You should see several Addivox files: `Addivox.app`, `Addivox.component`, `Addivox.clap` and `Addivox.vst3`.

You do not need to install every file. `Addivox.app` is the standalone version, which runs by itself and also contains the Audio Unit v3 plugin. The other files are additional plugin formats, which are loaded inside a DAW such as Logic Pro, GarageBand or Ableton Live.

These instructions also apply to the [Demo version](#) of Addivox. For the Demo version, the filenames use `AddivoxDemo` instead of `Addivox`, for example `AddivoxDemo.app` and `AddivoxDemo.vst3`.

Standalone app

Copy `Addivox.app` to your Mac's main `/Applications` folder. In Finder, this is usually shown as **Applications** in the sidebar. macOS may ask for an administrator password when you copy the app there. You can then open Addivox from Launchpad, Spotlight or Finder.

If macOS shows a warning the first time you open Addivox, try right-clicking `Addivox.app` and choosing **Open**. macOS may then ask you to confirm that you want to open it.

Audio Unit v3

The Audio Unit v3 version of Addivox is inside `Addivox.app`. This is normal for AUv3 plugins on macOS: the app acts as a container for a small app extension, and macOS makes that extension available to compatible DAWs.

To install the AUv3 version, copy `Addivox.app` to the main `/Applications` folder. This is the system-wide Applications folder, not a folder named `Applications` inside your home folder. There is no separate AUv3 file to copy into `Library/Audio/Plug-Ins`. After installing the app, quit and reopen your DAW. If Addivox does not appear, open `Addivox.app` once, then quit and reopen your DAW again.

Do not remove `Addivox.app` after the AUv3 plugin appears in your DAW. If you move the app to a different location or delete it, macOS may no longer be able to find the AUv3 plugin.

Plugins

Copy the plugin files you need to the matching folder below. The `~` symbol means your home folder.

File	Format	Install location	Example DAWs
<code>Addivox.app</code>	Audio Unit v3	<code>/Applications</code>	Logic Pro, GarageBand, MainStage
<code>Addivox.component</code>	Audio Unit v2	<code>~/Library/Audio/Plug-Ins/Components/</code>	Logic Pro, GarageBand, Ableton Live, REAPER
<code>Addivox.vst3</code>	VST3	<code>~/Library/Audio/Plug-Ins/VST3/</code>	Ableton Live, Cubase, REAPER, Studio One
<code>Addivox.clap</code>	CLAP	<code>~/Library/Audio/Plug-Ins/CLAP/</code>	Bitwig Studio, REAPER

If you are not sure which plugin format to install, start with `Addivox.app` for Logic Pro, GarageBand or MainStage, and `Addivox.vst3` for most other modern DAWs. `Addivox.component` is mainly useful for older DAWs.

The `Library` folder inside your home folder is hidden by default. To open one of these folders in Finder:

1. Open Finder.
2. Choose **Go > Go to Folder...** from the menu bar.
3. Paste the install location from the table above.
4. Press Return.
5. Create the folder if it does not already exist, then copy the Addivox plugin file into it.

After copying plugins, quit and reopen your DAW. Some DAWs scan new plugins automatically. Others have a plugin manager or preferences page where you can rescan plugins. If you installed more than one format, your DAW may show Addivox more than once, for example as both an Audio Unit and a VST3.

Intel and Apple Silicon Macs

Addivox is built as a 64-bit macOS app/plugin and is intended to work on both older Intel Macs and newer Apple Silicon Macs. There is no separate Intel download or Apple Silicon download; use the same files from `Addivox_v1.0.0_macOS.zip`.

All of the included formats can be used on Intel or Apple Silicon Macs, provided your DAW supports that plugin format:

- `Addivox.app`
- `Addivox.component`
- `Addivox.vst3`
- `Addivox.clap`

On an Intel Mac, use Addivox normally. On an Apple Silicon Mac, Addivox can run natively in Apple Silicon DAWs. It can also be used from Intel-only DAWs running under Rosetta, provided the DAW supports the plugin format you installed.

Addivox requires macOS 10.13 High Sierra or newer. Very old 32-bit DAWs and 32-bit plugin formats are not supported.

2.4 Installation – Windows

Addivox for Windows is delivered as a zip file with a name like `Addivox_v1.0.0_Windows.zip`. After downloading it, double-click to unzip it. You should see several Addivox files: `Addivox.exe`, `Addivox.clap` and `Addivox.vst3`.

You do not need to install every file. `Addivox.exe` is the standalone version, which runs by itself. The other files are plugin formats, which are loaded inside a DAW such as Ableton Live, Cubase or REAPER.

These instructions also apply to the [Demo version](#) of Addivox. For the Demo version, the filenames use `AddivoxDemo` instead of `Addivox`, for example `AddivoxDemo.exe` and `AddivoxDemo.vst3`.

Standalone app

`Addivox.exe` is self-contained. Move it anywhere convenient and optionally create a shortcut to it. You can then open Addivox by double-clicking the executable or shortcut. Addivox for Windows is not code-signed, so Windows SmartScreen or antivirus software may warn you the first time you open it.

For live playing, the standalone application may need a low-latency Windows audio driver. ASIO is usually recommended when available. See [Which Windows audio driver type should I use?](#) for more details.

Plugins

Copy the plugin files you need to the matching folder below.

File	Format	Per-user install location	All-users install location	Example DAWs
<code>Addivox.vst3</code>	VST3	<code>%LOCALAPPDATA%</code> <code>\Programs\Common\VST3</code>	<code>C:\Program</code> <code>Files\Common</code> <code>Files\VST3</code>	Ableton Live, Cubase, REAPER, Studio One
<code>Addivox.clap</code>	CLAP	<code>%LOCALAPPDATA%</code> <code>\Programs\Common\CLAP</code>	<code>C:\Program</code> <code>Files\Common</code> <code>Files\CLAP</code>	Bitwig Studio, REAPER

For VST3, copy the complete `Addivox.vst3` directory, not just the file inside it. For CLAP, copy the `Addivox.clap` file.

If you are not sure which plugin format to install, start with `Addivox.vst3` for most modern DAWs. Use `Addivox.clap` if your DAW supports CLAP and you prefer that format.

After copying plugins, quit and reopen your DAW. Some DAWs scan new plugins automatically. Others have a plugin manager or preferences page where you can rescan plugins. If you installed more than one format, your DAW may show Addivox more than once, for example as both a VST3 and a CLAP plugin.

Windows compatibility

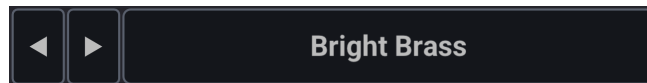
Addivox is built as a 64-bit Windows app/plugin. It is developed and tested on Windows 11 but should also work on Windows 10. Please try the [Demo version](#) before buying Addivox to ensure that it runs on your system.

Very old versions of Windows, 32-bit DAWs and 32-bit plugin formats are not supported.

3. Using Addivox

3.1 Main UI

Patch selector



The top bar shows the name of the currently loaded patch and provides controls to navigate between patches. The left and right arrows step through patches in the current group (you can also use your computer keyboard's left and right arrow keys), and the menu button opens the full patch browser where you can choose any factory or custom patch, save your work, or import and export patch files. See [Patches](#) for more detail.

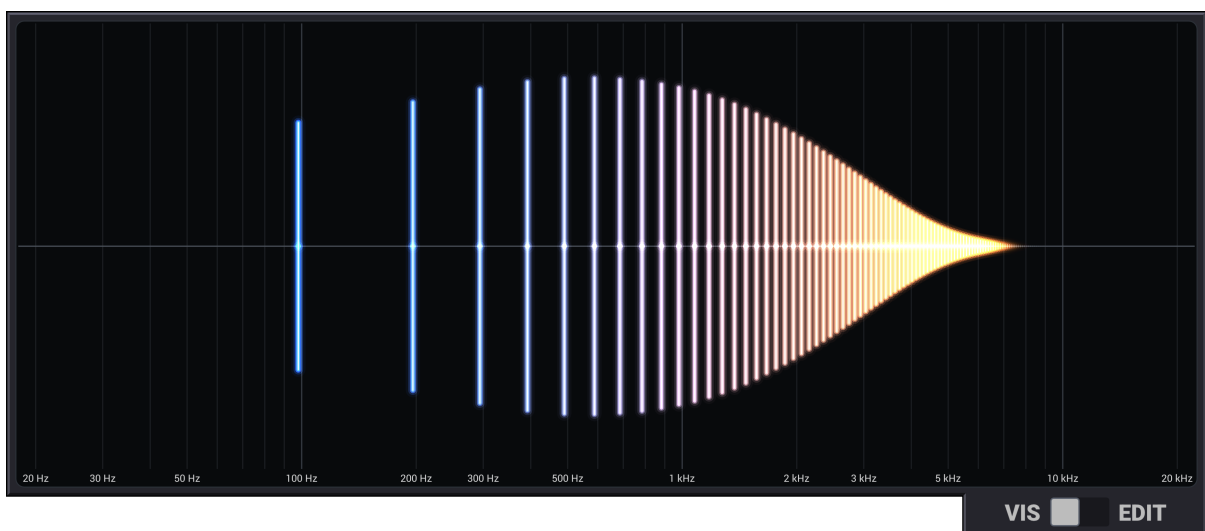
Settings menu

The gear icon in the top-right corner of the top bar opens the settings menu:

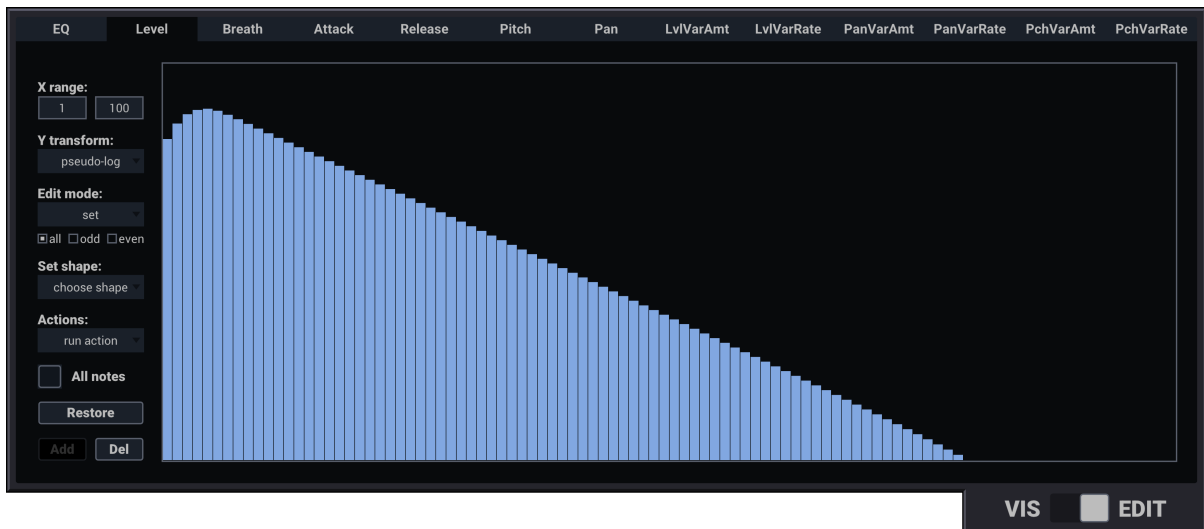
- **Visualizer Enabled** — toggles the harmonic visualizer in VIS mode; disabling it can reduce CPU usage on older hardware.
- **Breath CC** — selects the MIDI CC(s) used for breath/level control: CC 1 (Mod Wheel), CC 2 (Breath), CC 2 + CC 34 (High-Res Breath), CC 7 (Volume), CC 7 + CC 39 (High-Res Volume), CC 11 (Expression) or CC 11 + CC 43 (High-Res Expression).
- **Pitch Bend Range** — sets the pitch bend range to 1 semitone, 2 semitones, a fifth or an octave. This can also be set by right-clicking the pitch bend wheel.
- **Audio & MIDI Settings...** (*standalone app only*) — opens the system audio and MIDI device settings.
- **Reset Synth Settings to Defaults** — restores the first factory patch and resets transpose, tuning, pan shift, reverb, breath CC source, pitch bend range and the visualizer toggle to their defaults. Does not affect the Audio & MIDI Settings.

VIS / EDIT panel

The large panel in the middle of the UI has two modes, switched with the **VIS / EDIT** toggle in its bottom-right corner.



VIS mode shows a live bar chart of all 100 harmonics as you play. Bar heights update in real time to reflect the current output level of each harmonic, giving you a visual picture of the sound's harmonic content.



EDIT mode is where you shape the sound. It contains a set of tabs for editing per-harmonic parameters and the EQ curve:

- **EQ** — draws a frequency-response curve applied to the final sound. See [EQ](#).
- **Level, Breath, Attack, Release, Pitch, Pan**, and six variation tabs — control the individual behaviour of each of the 100 harmonic oscillators. See [Per-harmonic settings](#).

The keyboard at the bottom of the panel selects which key note you are editing in EDIT mode, and plays the corresponding note in VIS mode (see below).

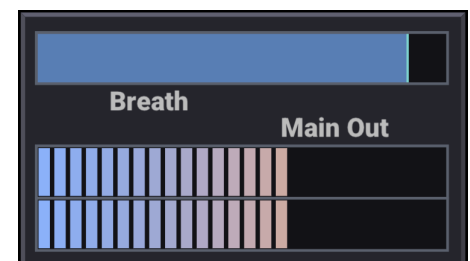
Control panels

Below the main panel are five rows of knobs and controls:

- **Pitch** — transpose, tuning, portamento.
- **Envelope** — global attack and release scales.
- **Variation** — global scales for level, pitch and pan variation depth and rate.
- **Output** — global level scale and pan offset.
- **Effects** — drive, tone, chorus and reverb.

Meters

The **breath meter** on the top shows the current incoming breath/CC signal level. The **output meter** on the bottom shows the stereo output level; red indicators at the top warn that the signal is above 0 dB and may clip when rendered or passed to later processing.



Keyboard



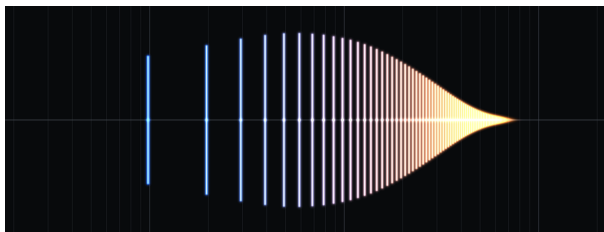
The keyboard along the bottom of the UI highlights the note currently being played. In VIS mode you can click keys to play notes directly (at full breath). In EDIT mode, clicking a key selects which key note's settings are displayed in the editor; if no key note exists at that note, the editor shows the interpolated values for that pitch. See [Key Notes and Interpolation](#).

The pitch bend wheel to the left of the keyboard bends the pitch of the currently held note. It follows pitch bend messages sent from the wind controller. Right-click the wheel to set the pitch bend range.

3.2 Patches

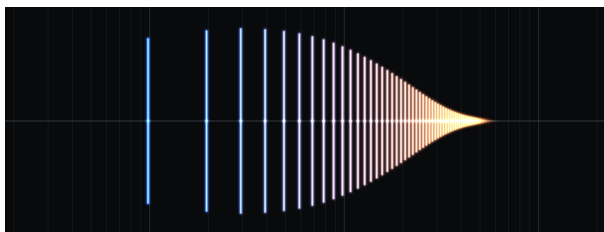
Factory patches

Addivox ships with 13 factory patches:



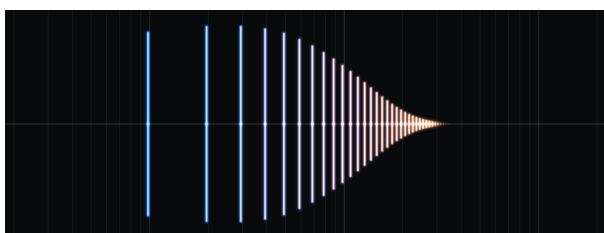
Bright Brass

A clean bright brassy sound, ranging from tuba-like to trombone-like to trumpet-like. Has a smoothly tapering Level curve, and for lower notes, the peak is at a higher harmonic number. No formants.



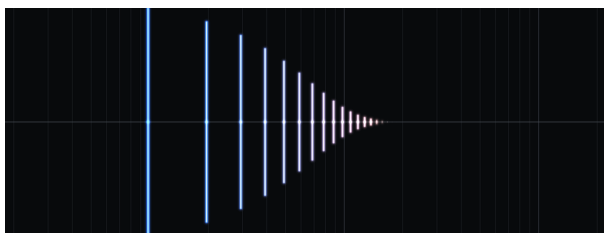
Warm Brass

Similar to Bright Brass, except the Level curve peaks sooner and tapers faster, giving fewer high harmonics.



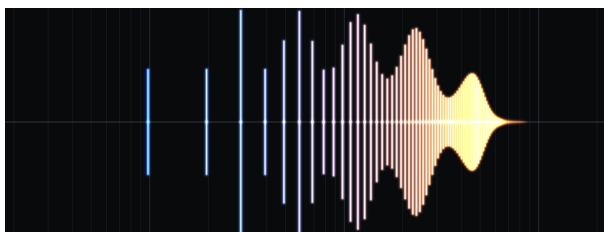
Mellow Brass

Similar to Warm Brass, except the Level curve peaks sooner and tapers faster, giving even fewer high harmonics.



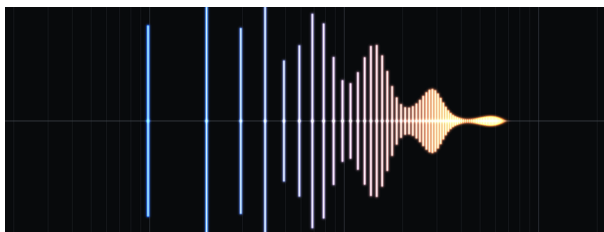
Dark Brass

Similar to Mellow Brass, except the Level curve peaks sooner (first harmonic is always strongest) and tapers faster, giving even fewer high harmonics.



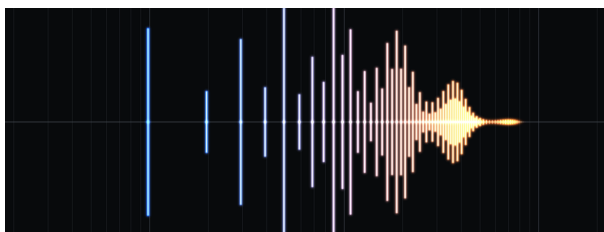
Bright Reed

A bright woodwind-like sound with strong formants across the entire spectrum.



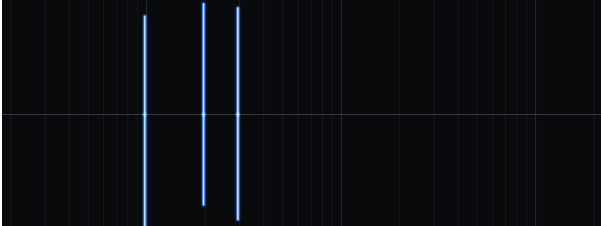
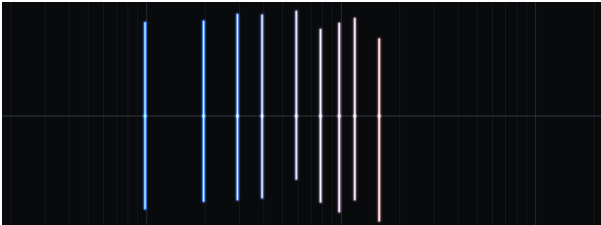
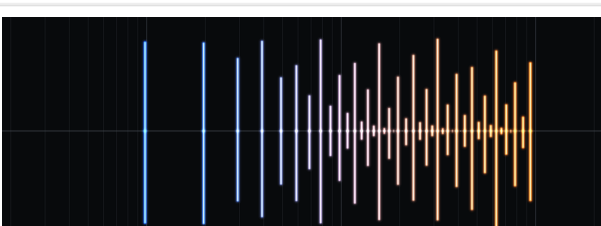
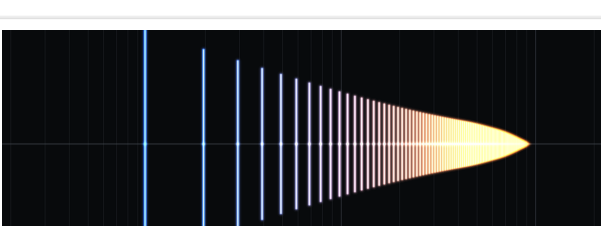
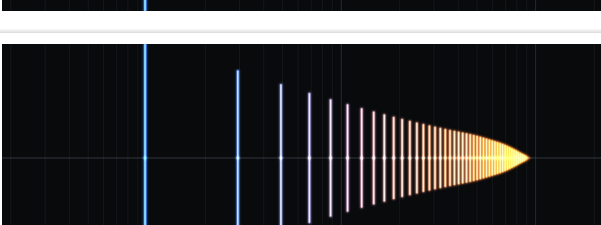
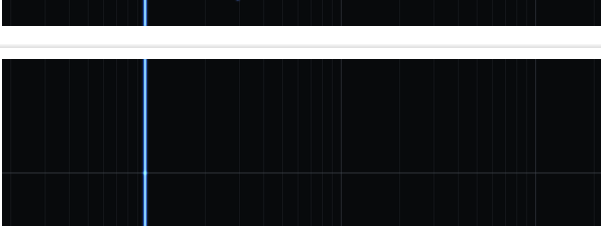
Warm Reed

Similar to Bright Reed, but the Level curve peaks sooner giving weaker high harmonics and with different formant peaks.



Hollow Reed

Similar to Warm Reed, but with an odd-harmonic bias for a clarinet-like sound and with different formant peaks.

	Tonewheel Organ 888 Inspired by a Hammond organ with drawbars set to 888000000. Contains a generous amount of level, pan and pitch variation to make the notes sound more lively.
	Tonewheel Organ Full Inspired by a Hammond organ with drawbars set to 888888888. Contains a generous amount of level, pan and pitch variation to make the notes sound more lively.
	Full Pipe Organ Has harmonics at equal levels across the octaves for a big full-spectrum sound. Contains a bit of level, pan and pitch variation.
	Simple Saw Level curve follows a $1/h$ shape, approximating a sawtooth wave. The top harmonics taper to zero to prevent a high-pitched whine sound at full breath for low notes.
	Simple Square Level curve follows a $1/h$ shape, odd harmonics only, approximating a square wave. The top harmonics taper to zero to prevent a high-pitched whine sound at full breath for low notes.
	Simple Sine Just a single oscillator at the fundamental. As simple as it gets!

Custom patches

You can create and save your own patches from within Addivox. Custom patches are stored as TOML text files in Addivox's patches folder:

- **macOS:** `~/Library/Application Support/Addivox/Patches/`
- **Windows:** `%LOCALAPPDATA%\Addivox\Patches\`

For the demo version, these folders are named `AddivoxDemo` instead of `Addivox`.

Some plugin formats run in a sandbox (for example, AUv3 on macOS). A sandboxed plugin keeps its own separate patches folder, so patches saved there won't automatically appear in the standalone app or in other plugin formats. If a patch seems to be missing, this is the most likely reason — use the "Show User Patches in Finder" command in the patch menu to see exactly which folder that instance of Addivox is using, and copy patch files between folders as needed.

Patches placed in subdirectories of this folder appear in their own named group in the patch menu, which you can use to organise larger collections.

Patch file format

Patches are plain-text [TOML](#) files. Here is a minimal example of a single-key-note patch:

```
format_version = 1
name = "My Patch"

[voice_settings]
levelScale = 1.0
attackScale = 1.0
releaseScale = 1.0
levelVariationAmplitudeScale = 0.0
levelVariationRateScale = 1.0
pitchVariationAmplitudeScale = 0.0
pitchVariationRateScale = 1.0
panVariationAmplitudeScale = 0.0
panVariationRateScale = 1.0
portamentoTimeAtCC5MinSec = 0.001
portamentoTimeAtCC5MaxSec = 0.025

[effects_settings]
drive = 0.0
tone = 0.0
chorus = 0.0

[[key_notes]]
midi_note = 60
note_name = "C4"
level      = [0.9, 0.8, 0.7, ...]
breath_power = [1.0, 1.0, 1.0, ...]
attack     = [0.005, ...]
release    = [0.01, ...]
pitch      = [0.0, ...]
pan        = [0.0, ...]
level_variation_amplitude = [0.25, ...]
level_variation_rate      = [1.0, ...]
pitch_variation_amplitude = [5.0, ...]
pitch_variation_rate      = [1.0, ...]
pan_variation_amplitude   = [0.25, ...]
pan_variation_rate        = [1.0, ...]
eq_freq_hz = []
eq_db      = []
```

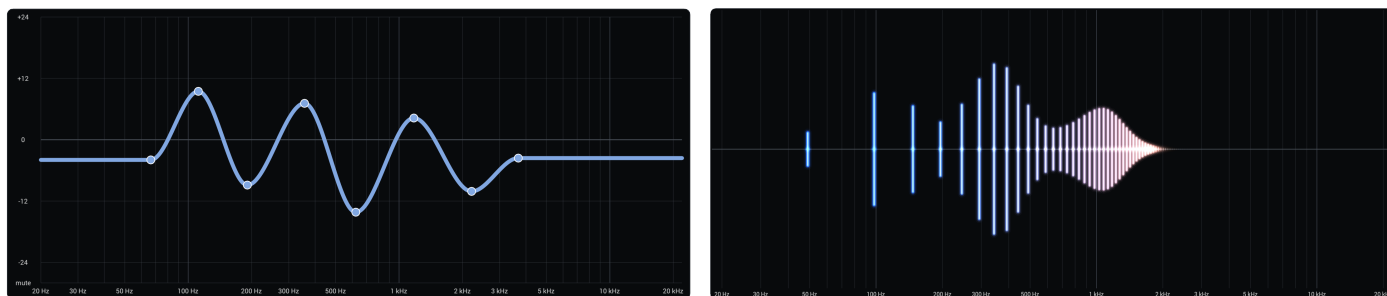
Each per-harmonic array contains exactly 100 values (one per harmonic). A patch with multiple key notes has one `[[key_notes]]` section for each. An optional `[all_key_notes]` section stores parameters that are locked in sync across all key notes (see [Key Notes and Interpolation](#)).

Because patches are plain text, you are welcome to open and edit them in any text editor. As long as the file follows the format above and has a `.toml` extension, Addivox will load it correctly.

3.3 EQ

The EQ tab in the editor lets you draw a frequency-response curve that is applied to the synthesised sound. The curve is defined by a set of control points — you can drag existing points to reshape it, and add/remove points by double-clicking. The curve is smoothly interpolated between points using a monotone spline, and it is flat (0 dB) outside the leftmost and rightmost points.

Unlike the per-harmonic settings, the EQ curve is anchored to absolute frequencies rather than to the pitch of the note. This means a peak at 800 Hz stays at 800 Hz regardless of whether you are playing a low note or a high note.



Formants

A **formant** is a resonant peak in the frequency spectrum — a band of frequencies that the sound emphasises. Formants are what give acoustic instruments and the human voice their characteristic timbre. For example, the difference between a saxophone and an oboe comes in part from where their formants sit in the frequency spectrum.

In real instruments, formants arise from the physical resonances of the instrument. Because these shapes are fixed, the formants stay at roughly the same frequencies even as you play higher or lower notes. Addivox's EQ works the same way: keeping the curve fixed in frequency lets you shape formants that behave like those of a real instrument.

Adding one or more EQ peaks can dramatically change the timbre of a patch. The **All notes** toggle (see [Key Notes and Interpolation](#)) is particularly useful here — it lets you set a single EQ curve that applies across all key notes at once, which is usually what you want for formants.

3.4 Per-harmonic Settings

In EDIT mode, the main panel contains a row of tabs. Each tab shows a bar chart with one bar per harmonic (up to 100). You can drag bars up and down to set values, use the **Set shape** dropdown to apply a preset curve, or use the **Run action** dropdown to apply transformations like scaling or normalising. Many actions in these dropdowns have a keyboard shortcut, shown as a letter in brackets next to the action name.

The **X range** controls at the bottom let you zoom in on a subset of harmonics, making it easier to edit the higher harmonics where bars would otherwise be very narrow.

The **All notes** toggle, when enabled, locks that parameter in sync across all key notes. See [Key Notes and Interpolation](#).

Level

Controls the amplitude of each harmonic at full breath. This is the primary control for shaping the tonal character of a patch — setting the relative strength of each harmonic determines whether the sound is bright or mellow, thin or full.

Breath

Controls per-harmonic breath sensitivity (the `breath_power` exponent). At a value of 1, a harmonic's level scales linearly with breath. Higher values make the harmonic require proportionally more breath before it becomes prominent — effectively hiding quieter harmonics until the player blows harder. Lower values make a harmonic appear at low breath levels.

Attack / Release

Controls how quickly each harmonic can increase (**Attack**) or decrease (**Release**) in level in response to breath changes. Higher attack values make a harmonic bloom more slowly; higher release values make it fade more slowly. These are scaled globally by the [envelope controls](#).

Pitch

A static pitch offset in cents for each harmonic, added on top of its natural harmonic frequency. Small detuning of individual harmonics can add warmth and movement to the sound.

Pan

The stereo position of each harmonic, from -1 (fully left) to +1 (fully right). Spreading harmonics across the stereo field can add width and richness to the sound.

Variation tabs

Six tabs control continuous slow modulation of level, pitch, and pan. Each has an **Amount** tab (depth of the modulation for that harmonic) and a **Rate** tab (speed of the modulation). These are scaled globally by the [variation controls](#).

- **LvlVarAmt** / **LvlVarRate** — level variation
- **PchVarAmt** / **PchVarRate** — pitch variation
- **PanVarAmt** / **PanVarRate** — pan variation

Y-axis transformations

Addivox's edit tabs let you view the per-harmonic settings as a bar chart. The y-axis transform controls how a bar's height maps to its actual value.

Linear is the most intuitive: a bar twice as tall represents twice the value. But this makes fine adjustments near zero difficult — small values are squashed into a thin sliver at the bottom of the chart.

The **Square root** and **Pseudo-log** transforms both stretch the lower end of the scale, giving you more visual room for near-zero values so small adjustments are easier to make. Square root gives a moderate stretch; Pseudo-log stretches more dramatically.

Pseudo-log is particularly well suited to audio levels. Our ears perceive loudness on a logarithmic scale — this is why volume is measured in decibels. Using Pseudo-log for the "Level" edit tab means equal changes in bar height correspond to roughly equal changes in perceived loudness. (A pure [log transform](#) has one problem: it can never actually reach zero. Pseudo-log is a modified version that behaves like a log for most of its range but allows a bar to be set all the way to zero at the bottom.)

All of these transform options are purely visual — they do not change the sound produced by Addivox.

3.5 Key Notes and Interpolation

A **key note** is a MIDI note at which you define an explicit set of per-harmonic values. Addivox uses key notes to let a patch sound different in different registers.

Single key note

If a patch has only one key note, all played notes share the same per-harmonic settings. The patch sounds the same regardless of octave, except for the fundamental pitch of each harmonic (which always scales with the played note).

Multiple key notes

If a patch has two or more key notes, Addivox interpolates smoothly between them. When you play a note that falls between two key notes, each per-harmonic value is a weighted blend of the two neighbouring key notes based on how close the played note is to each one. Notes below the lowest key note use the lowest key note's settings; notes above the highest use the highest.

This allows a single patch to have a heavy, lower-harmonic timbre in the bass register and a brighter, thinner timbre in the upper register — matching how real acoustic instruments change character across their range. The factory brass patches use key notes at each octave (C1 through C8) for exactly this reason.

"All notes" toggle

Each per-harmonic parameter and the EQ curve have an **All notes** toggle in the editor. When enabled, that parameter is kept in sync across every key note: editing it on one key note instantly updates all others. This is useful for settings that should not vary with register — formant EQ curves are the primary example, since real instrument formants stay fixed in frequency regardless of which note is played.

3.6 Pitch settings

Addivox is a **monosynth** — it plays one note at a time.

Transpose shifts every played note by a fixed number of semitones. The keyboard display still highlights the incoming MIDI note, but the sounded pitch is shifted by the transpose amount. This control is not patch-tied and keeps its value when you switch patches.

Tuning applies a global pitch offset in cents (hundredths of a semitone) to all harmonics. Like transpose, this is not patch-tied.

Portamento controls smooth pitch glides between notes. The rate of the glide is set by MIDI CC5 (typically the breath controller's bite sensor on a wind controller). The **Min** and **Max** values set the glide time range: **Min** is the glide time when CC5 is 0 (usually instant or very fast), and **Max** is the glide time when CC5 is at full value.



3.7 Global settings

Envelope

Attack and **Release** are scale multipliers applied on top of each harmonic's individual attack and release times (set in the [per-harmonic editor](#)).

- **Attack** — higher values slow down how quickly harmonics bloom in when a note starts.
- **Release** — higher values let harmonics ring out longer after a note ends.



Variation

Variation adds continuous slow modulation to the sound, giving it movement and life. Each of the three modulation targets — **level**, **pitch**, and **pan** — has two controls:

- **Amount** — scales the depth of that modulation across all harmonics.
- **Rate** — scales the speed of that modulation across all harmonics.

The per-harmonic variation depths and rates (set in the [per-harmonic editor](#)) are multiplied by these global scales, so setting an Amount to 0 silences that variation entirely. Level variation gives a subtle tremolo-like effect; pitch variation adds vibrato; pan variation slowly moves harmonics left and right in the stereo field.



Output

- **Pan** — shifts all harmonics left or right in the stereo field. Unlike most controls, this is not tied to the current patch and will keep its value when you switch patches.
- **Level** — sets the overall output volume of the synth.



3.8 Effects

Addivox includes a small set of effects for shaping the final sound:

- **Drive** is a soft saturation effect. It blends in an oversampled tanh-style waveshaper, with more gain, bias and smoothing as the control increases. Low settings add warmth; high settings become more obviously distorted.
- **Tone** is a tilt EQ. Negative values make the sound darker by emphasising lower bands, while positive values make it brighter by emphasising higher bands.
- **Chorus** is an 8-voice stereo chorus. It uses short modulated delays, filtered wet signal and spread-out panning to widen and thicken the sound.
- **Reverb** adds synthetic room ambience. Low settings are shorter and more early-reflection focused. High settings add longer pre-delay, a longer tail, more modulation and a wider late field.



Unlike the rest of Addivox, which operates on a per-harmonic oscillator basis, these effects are applied to the final stereo waveform after synthesis. They are applied in the order listed above: drive first, then tone, then chorus, then reverb. Using effects (any setting other than 0) will increase Addivox's CPU usage.

Drive, tone and chorus are saved in patches (see [Patches](#)). Reverb is a persistent control, so it holds its value when patches change.

These are simple one-knob effects, intended mainly to give the standalone versions of Addivox a finished sound. When using Addivox as a plugin in a DAW, you may prefer to leave them off and use dedicated effects plugins instead. For that reason, reverb defaults to 50% for standalone versions of Addivox and 0% for plugin versions of Addivox.

4. Support

4.1 Videos

The Addivox YouTube channel: youtube.com/@Addivox

Videos from others

The resources listed below aren't affiliated with Addivox, but they contain helpful information about software synthesizers and wind controllers:

- [How to use SWAM instruments with your wind synth](#) from [iSax.Academy](#): focused on SWAM instruments, but many topics covered (MIDI, buffer size, Bluetooth, etc) also apply to Addivox.
- [Why MainStage is Great for Digital Wind Instruments](#) from [Stef Haynes](#): walks through a MainStage concert setup and shows how MIDI CC can control a synth. Uses Apple's ES1, but many concepts apply to Addivox too.
- [Find Your Voice - Playing Synth Sounds](#), a podcast from [Aerophone Academy](#): discusses embracing synth sounds on their own terms, rather than treating them as emulations of acoustic instruments.

4.2 FAQ and Miscellaneous Tips

What are the system requirements for Addivox?

I can't give specific minimum specs, since performance depends on your DAW, buffer size and whatever else you're running alongside Addivox. But as a couple of data points, it runs fine on an M1 MacBook Air (2020) and on an Intel Core i3-8109U (2018), and it doesn't use much memory. If your hardware is older or less powerful, you can reduce CPU usage by disabling the visualizer in the [settings menu](#). The best way to know for sure is to try before you buy: download the [demo version of Addivox](#) and confirm it runs well on your system before purchasing.

Which Windows audio driver type should I use?

This only applies to the standalone Windows application. If you use Addivox as a VST3 or CLAP plugin, your DAW handles the audio driver.

On a new Windows installation, Addivox starts with DirectSound unless you choose something else. You can change the driver type in the standalone app's Audio & MIDI Settings.

For real-time playing, **ASIO** is usually the best choice. If your audio interface has a native ASIO driver from its manufacturer, use that. Native ASIO drivers usually give the lowest latency and most reliable performance. If you do not have a native ASIO driver, [ASIO4ALL](#) can be worth trying.

DirectSound is the basic Windows option. It is useful as a fallback and often works without extra setup, but it may have too much latency for live playing.

WASAPI is another built-in Windows option. Addivox supports WASAPI shared mode, which allows other applications to keep using the audio device at the same time. On some systems it may work well, but it is not always lower-latency than DirectSound.

Clipping audio on macOS and Windows

Addivox can produce floating-point audio above 0 dBFS, especially if a patch has high per-harmonic levels, is played at full breath, uses **Level variation** or if the global **Level** knob is turned up. The **Main Out** meter shows peak output level; if it reaches the red range, the signal is at or above 0 dBFS and may clip somewhere after Addivox.

Whether or not an output level > 0 dBFS will cause clipping distortion seems to depend on the platform and audio driver. In my experience, macOS (Core Audio) and Windows WASAPI are tolerant of high levels, and I do not hear distortion when Addivox exceeds 0 dBFS. But with Windows DirectSound and Windows ASIO, I do hear clipping distortion.

The safest habit is to watch the **Main Out** meter during playing, and if you see it reach the red range, reduce the overall level, either via the [per-harmonic](#) Level tab or the [global](#) Level knob.

Is a Pro Tools (AAX) plugin available?

No, Addivox is not currently available as an AAX plugin for Pro Tools.

iPlug2 can build AAX plugins, so this is possible in principle. However, public Pro Tools builds require AAX plugins to be signed through Avid/PACE's signing system before they can be loaded. I also do not own Pro Tools set up for proper testing, so I would not be able to test any AAX builds I made.

Can Addivox be played with a keyboard?

Sure, though Addivox is built around breath control, so played from a plain keyboard with no breath input, it will sound flat. The easiest fix is a mod wheel (CC 1), since many keyboards have one built in. A breath controller or an expression/volume pedal linked to CC 2, CC 7 or CC 11 works too. I haven't tried playing Addivox with a keyboard myself, so your mileage may vary. Velocity and aftertouch are not currently supported as a breath source — if that's a feature you'd like, please [let me know](#). One more thing to keep in mind: Addivox is a monosynth, so it can only play one note at a time, which rules out chords.

Can Addivox mimic real instrument sounds?

Perhaps, with some careful fine-tuning of the patches, but this is not what Addivox was designed for. If your goal is to mimic a real instrument with your wind controller, then something like the [SWAM instruments](#) would be a better choice.

Can Addivox produce breathy flute sounds?

No, it cannot. Addivox's synthesis is based on summing harmonic sine waves, and it does not have the ability to create noise (non-pitched sound) in its synthesised output. If you are interested in breathy flute sounds, I have found [Respiro](#) (a physical-modelling synthesizer) to excel at this.

Why additive synthesis?

There are many ways to synthesize sound, so why is Addivox built around additive synthesis? The biggest reason is control: independent level, breath response, attack/release, pitch, pan and variation for each of the 100 harmonics. This allows for some cool effects that are not easy with other synthesis methods, e.g. the pitch variation I use in the organ-style presets.

Additive synthesis also makes it easy to implement [formants](#). During synthesis, each harmonic is scaled based on its frequency and the [EQ curve](#), rather than rendering a waveform first and processing it with an EQ filter afterwards, which is less precise.

Since Addivox always knows the exact frequency of every harmonic, it can simply skip rendering any harmonic above the [Nyquist frequency](#), which solves aliasing. The same certainty is what makes the visualization possible too: it shows the exact level of all 100 harmonics in real time, without the smearing of an FFT-based spectrum analyser.

The main cost is CPU usage: 100 explicit oscillators per note is more expensive than a typical synth voice. But Addivox is a monosynth (one note at a time), so that cost doesn't scale with polyphony.

What was the inspiration for Addivox?

Like many who play wind controllers, I enjoyed the brassy sounds of the [EVI-NER](#) synthesizer. However, I had a couple issues with EVI-NER. First of all, its sound was a bit too synthesizery, especially in the low end. I wanted something similar to a brass sound over the full range, from tuba-like in the low range to trombone-like in the mid range to trumpet-like in the upper range. Second, while I was able to register my copy of EVI-NER and get my serial number, I have seen many reports online of people who could not, so it seems that EVI-NER may not be a reliable option for the windsynth community.

Addivox solves the first issue by allowing for multiple [key notes](#) in its patches. This allows you to define different sounds in different registers, creating patches that smoothly span many octaves. And Addivox solves the second issue by being source-available (see [License](#)) and not containing any DRM like serial numbers or activation codes.

Finally, while I did not start developing Addivox with formants in mind, during development I learned how important they can be for shaping the timbre of a sound. So I now consider the ability to apply formant EQ filters to be one of Addivox's key features (see [EQ](#)).

4.3 Support and feedback

I did my best to make Addivox work across different computers, operating systems, audio drivers, hardware and DAWs. But there are many possible setups, so my apologies if I missed something and Addivox does not work properly for you – please let me know, and I'll troubleshoot the best I can. You can also check the [Known issues](#) page to see if your problem is already known. And don't forget to try the free [demo version](#) before you buy!

If you find a new bug, have a feature request, or want to ask a question, please email: addivox.support@gmail.com

If you have a GitHub account, you can also use the [Addivox GitHub issues page](#).

4.4 Release notes

Availability of past versions

When new versions of Addivox are released, they will be made available through Lemon Squeezy to both new and existing customers.

New versions of Addivox are of course intended to improve stability and functionality, but there may be cases where an older version works better for your setup. Previous versions of Addivox are therefore available to customers on request. If you need an older version, please [email me](#) using the email address used for purchase, or provide your Lemon Squeezy order number.

Addivox v1.0.1 – 11 July 2026

Some bug fixes:

- Fixed the AUv3 plugin failing to load correctly in some hosts.
- Fixed the AUv3 plugin's window opening at the wrong size.
- Fixed Objective-C class collisions between the full and demo versions.
- Fixed a rare crash caused by a state-handling race condition.
- Fixed a rare crash when closing the plugin window while a menu was open.

Full version:

- [Addivox_v1.0.1_macOS.zip](#)
SHA-256 = 60ed98cea0c76010156d44fe75675bbec99d5bedd78551ec9e74dea07d6fb34c
- [Addivox_v1.0.1_Windows.zip](#)
SHA-256 = 3fe84773ff661b6a76b5b98ece27a0df001d243d5d0a917b2f4a2450fcf234bf

Demo version:

- [AddivoxDemo_v1.0.1_macOS.zip](#)
SHA-256 = 8e105f58ebb3f64579bc084cfcdc5f42c090bd03bd4c2f20d1139dbc506f61b4
- [AddivoxDemo_v1.0.1_Windows.zip](#)
SHA-256 = bbe731a8d8791a0becde740a7a2a22f39a51af5e2f1000250b65d60ffb631e55

Addivox v1.0.0 – 09 July 2026

The very first release of Addivox 🤪

Full version:

- [Addivox_v1.0.0_macOS.zip](#)
SHA-256 = 9311005a869175a3ff7e7ff67871d6dbf315b344b544449195ebe8e26b784a29
- [Addivox_v1.0.0_Windows.zip](#)
SHA-256 = 03591aca4a137fd9adf68509868935dd866dc8e74ea2544dff31a00c92371a00

Demo version:

- [AddivoxDemo_v1.0.0_macOS.zip](#)

SHA-256 = 8eb0308473a1035f51ad222ac4a4986a02cfac8c146fccca7bc23bdeec908460

- [AddivoxDemo_v1.0.0_Windows.zip](#)

SHA-256 = a7f1079319d8a000903cfef9009ea2338fb51d590abd4e5da9cf5c9ec5d14183

4.5 Known issues

WASAPI Exclusive is not available on Windows

The Windows standalone application supports DirectSound, ASIO and WASAPI shared mode, but it does not currently support WASAPI Exclusive mode. For low-latency real-time playing on Windows, ASIO is recommended when available. See [Which Windows audio driver type should I use?](#).

Non-ASCII audio device names display as mangled text

Audio device names containing non-ASCII characters (e.g. curly apostrophes, accented letters) can be corrupted in the Audio and MIDI settings dialog. Example: `Ryan's iPhone Microphone` → `RyanÖs iPhone Microphone`. This bug is cosmetic only – a device with a mangled name still works correctly for audio I/O.

AUv3 "View" size control doesn't scale the interface

In GarageBand and MainStage, AUv3 plugin windows have a "View" percentage control for resizing the window. For Addivox, this control changes the size of the window but not the size of the Addivox interface itself, so setting it to anything other than 100% will crop the interface or leave blank space around it. Leave View set to 100% for the interface to display correctly.

4.6 License

Addivox is available [on GitHub](#), but Addivox is not open-source software. It is source-available under a [custom license](#), which aims to let curious users and developers inspect, build and privately modify Addivox while I retain redistribution rights. The license does not restrict music, recordings or patches made with Addivox.

Plain-language summary of the license terms

You may:

- if you have purchased Addivox, install and use it on all devices that you personally own or control
- use Addivox, including your own private builds and modifications, to make music and other output, including commercial work
- create, share and sell patches for Addivox
- read, download and change the Addivox source code for your own use
- build Addivox for yourself

You may not:

- share the Addivox source code or builds with others
- let other people use your purchase, download access or copy of Addivox
- share modified versions of Addivox
- sell or redistribute Addivox itself

This summary is for convenience only and is not part of the license. The full license text is available [here](#).